

MLA03 -- REINFORCEMENT LEARNING ASSIGNMENT

Department:	AI & ML
Programme:	FACULTY OF ENGINEERING
Course Code & Course Name:	MLA03 & REINFORCEMENT LEARNING
Academic Year / Batch:	2026-2027
Faculty Name:	Dr.G.A.SENTHIL
Assignment Title:	Design and Analysis of an RL-Based Patient Health Monitoring and Treatment Recommendation System Using Markov Decision Process and Bellman Equations
Date of Issue:	31-08-26
Date of Submission:	03-09-26
Maximum Marks:	100
Course Outcome(s) - CO:	CO1-Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. CO2 - Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies. CO3 - Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. CO4-Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. CO5-Apply hierarchical, meta-learning, and multi-agent RL approaches to solve complex decision-making problems.
Bloom's Taxonomy Level:	BL3 (Apply) - Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. BL4 (Analyse) - Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. BL5 (Evaluate) - Model-based reinforcement learning approaches for prediction, planning, and policy optimization. BL6 (Create) - Develop an MDP-based solution for an intelligent traffic signal system by defining the state space, action space, reward function, transition model, and optimal policy.
SDG Mapping (SDG 1-17):	SDG 3 -- (Good Health & Well-Being) Improves patient outcomes through data-driven, adaptive treatment recommendations. SDG 9 -- (Industry, Innovation & Infrastructure) Applies AI/RL to build intelligent clinical decision-support infrastructure. SDG 10 -- (Reduced Inequalities) Supports consistent, evidence-based treatment guidance that can extend specialist-level decision support to under-resourced settings.
Industry / Societal Relevance:	The assignment addresses the growing need for AI-driven clinical decision support by developing an RL-based patient monitoring and treatment recommendation system. The solution can help clinicians personalize treatment, respond earlier to deteriorating patients, and reduce adverse outcomes, while remaining subject to clinician oversight and regulatory approval.

Question

Analyse and design an RL-based patient health monitoring and treatment recommendation system using Markov Decision Process and Bellman equations. Model patient health conditions as states, treatment interventions as actions, and clinical outcomes as rewards, and implement an optimal policy to recommend appropriate treatment strategies based on changing patient conditions. Analyse the learned policy using treatment effectiveness, recovery rate, and cumulative reward.

Assignment Problem / Challenge

Healthcare providers must continuously decide how to treat patients whose health status changes over time in response to illness progression and to the treatments already given. Fixed, rule-based treatment protocols do not adapt well to the sequential, uncertain nature of patient recovery, where today's treatment choice affects tomorrow's health state and future treatment options. The assignment challenges students to design and analyse an autonomous Reinforcement Learning-based system that models patient health monitoring and treatment recommendation as a Markov Decision Process, uses the Bellman equations to derive an optimal treatment policy, and evaluates that policy using clinically meaningful measures: treatment effectiveness, recovery rate, and cumulative reward.

Problem Statement

Modern healthcare systems generate continuous streams of patient data - vital signs, laboratory results, symptom severity, and treatment history - that must be translated into timely, appropriate treatment decisions. A treatment chosen today changes the probability of the patient's health state tomorrow, so treatment selection is inherently a sequential decision-making problem rather than a single-shot classification problem. There is a need to design an RL-based patient health monitoring and treatment recommendation system that represents patient conditions as states, treatment interventions as actions, and clinical outcomes as rewards within a formal Markov Decision Process, and that uses the Bellman optimality equation (via value iteration / policy iteration) to compute an optimal treatment policy that maximizes long-term patient well-being while remaining safe, transparent, and subject to clinician oversight.

Requirements and Constraints

Requirements

- Continuously monitor patient vital signs, laboratory values, symptom severity, and treatment/medication history.
- Define the RL environment using appropriate states, actions, transition dynamics, reward function, and policy.
- Formulate the Bellman Expectation Equation and Bellman Optimality Equation for the patient-treatment MDP.
- Implement Dynamic Programming (value iteration and/or policy iteration) to compute the optimal treatment policy.
- Recommend treatment actions that adapt to changing patient conditions over successive monitoring intervals.
- Evaluate the learned policy using treatment effectiveness, recovery rate, and cumulative reward.
- Present the results using state-transition diagrams, value tables, policy tables, and convergence plots.

- Incorporate patient safety, clinical oversight, and ethical considerations into the system design.

Constraints

- Patient physiology is stochastic and only partially captured by measurable vitals and labs (partial observability).
- Clinical data is limited, imbalanced, and subject to privacy regulations (e.g., HIPAA/GDPR).
- Incorrect treatment recommendations can cause real patient harm, so exploration must be constrained and clinician sign-off retained.
- The reward function must correctly balance short-term symptom relief against long-term recovery and treatment cost/side effects.
- The system must generalize across patients with different comorbidities without becoming unsafe for atypical cases.
- Computational resources may limit real-time re-planning for very large, continuous state spaces.

What is the problem?

A patient health monitoring and treatment recommendation system must be developed that can (a) continuously observe patient vitals, laboratory results, and symptoms, (b) formulate treatment selection as a Markov Decision Process, (c) learn an optimal treatment policy using Dynamic Programming and the Bellman equations, (d) adapt treatment recommendations as the patient's condition changes over time, (e) balance recovery speed against treatment cost, side effects, and risk, and (f) present its recommendation and rationale to the clinician for final approval.

Expected Outcomes

A clearly defined MDP formulation containing states, actions, transition dynamics, reward function, and policy for patient treatment.

A Bellman-equation-based Dynamic Programming solution (value iteration / policy iteration) that computes the optimal state-value function $V^*(s)$, action-value function $Q^*(s,a)$, and optimal policy $\pi^*(s)$.

A worked, illustrative patient-treatment environment demonstrating how the learned policy changes with patient severity.

Quantitative evaluation of the learned policy using treatment effectiveness, recovery rate, and cumulative reward.

A working, documented end-to-end architecture connecting patient monitoring data, state representation, the RL agent, treatment-action selection, and clinical-outcome feedback.

Available Information / Data

- A synthetic or de-identified patient dataset containing vital signs (heart rate, blood pressure, SpO2, temperature), laboratory values, symptom severity, diagnosis/stage, and treatment history.
- Historical or simulated time-series patient monitoring data representing disease progression and recovery/deterioration under different treatments.
- Clinical outcome labels (e.g., recovered, still under treatment, deteriorated, adverse event) associated with each monitoring interval.
- A simulated patient-treatment environment (illustrative transition probabilities and rewards) for computing and evaluating the optimal policy.

Assumptions

- Patient vitals, labs, and symptom scores are assumed to be sufficiently informative to summarise the clinically relevant health state (the Markov property is assumed to approximately hold at the chosen level of state abstraction).
- Monitoring data may contain noise, but is assumed sufficient for the RL agent to make clinically meaningful decisions between monitoring intervals.
- Transition probabilities between health states, conditioned on the treatment given, are assumed to be stationary over the planning horizon used for policy computation.
- The reward function is assumed to correctly encode the desired balance between recovery, symptom control, treatment cost/side effects, and risk of deterioration.
- Real-world deployment requires clinician supervision and override authority, even though the RL system computes an optimal policy autonomously.

Reinforcement Learning Formulation: MDP and Bellman Equations

Markov Decision Process for Patient Treatment

For the assignment “Design and Analysis of an RL-Based Patient Health Monitoring and Treatment Recommendation System,” the patient-treatment environment is modelled as a Markov Decision Process:

$$MDP = (S, A, P, R, \gamma)$$

where S is the set of patient health states, A is the set of treatment actions, P is the state-transition probability function, R is the reward function, and $\gamma \in [0,1)$ is the discount factor that determines how strongly future clinical outcomes are weighted relative to immediate ones.

State Space

The state st summarises the patient's clinically relevant condition at monitoring interval t :

$$st = [V_t, L_t, H_t, D_t, T_t, R_t]$$

- V_t = vital-sign profile (heart rate, blood pressure, SpO2, temperature)
- L_t = laboratory/biomarker readings
- H_t = overall health-severity level (e.g., Stable, Mild Risk, Moderate Risk, Critical)
- D_t = disease/diagnosis stage or symptom score
- T_t = current treatment/medication in progress
- R_t = response trend since the previous interval (improving / stable / worsening)

For the small illustrative environment developed in this assignment, the severity level H_t is used as a compact discrete state variable with six states:

$$S = \{ \text{Stable, Mild Risk, Moderate Risk, Critical, Recovered, Deceased} \}$$

Recovered and Deceased are absorbing terminal states: once reached, no further treatment action changes the state.

Action Space

The action a_t represents the treatment intervention chosen at interval t :

$$A = \{ a_0, a_1, a_2, a_3 \}$$

- a_0 : No intervention / routine observation

- a1: Standard medication
- a2: Intensive monitoring with adjusted medication dosage
- a3: Hospitalisation / ICU-level intervention

Transition Dynamics

The transition function $P(s'|s,a)$ gives the probability that the patient moves to health state s' at the next interval, given current state s and treatment action a . Table 1 shows an illustrative transition matrix for the Moderate Risk state; transitions for other states follow the same pattern, with more aggressive treatments generally increasing the probability of improvement but carrying higher cost.

From: Moderate Risk, given action →	Mild Risk	Moderate Risk	Critical	Recovered
a0: No intervention	0.10	0.45	0.40	0.05
a1: Standard medication	0.35	0.45	0.15	0.05
a2: Intensive monitoring	0.45	0.35	0.08	0.12
a3: Hospitalisation / ICU	0.30	0.25	0.05	0.40

Table 1: Illustrative transition probabilities $P(s'|s,a)$ from the Moderate Risk state.

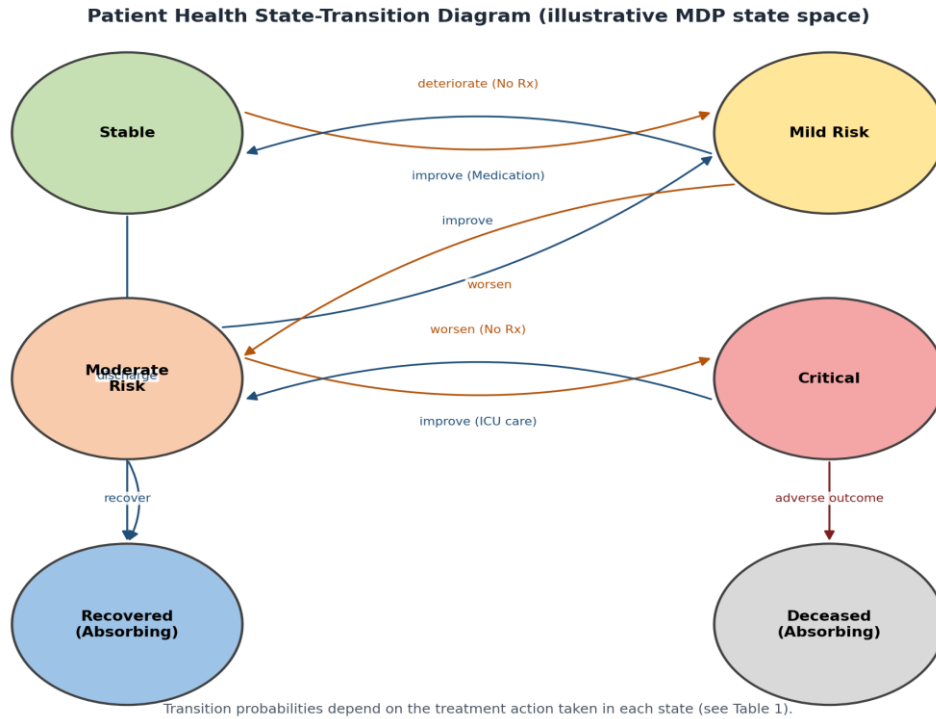


Figure 1: Patient health state-transition diagram (illustrative MDP state space).

Reward Function

The reward balances recovery, symptom relief, treatment cost, and risk:

$$R_t = \alpha \cdot C_t - \beta \cdot X_t - \gamma \cdot E_t - \delta \cdot P_t$$

- C_t = clinical-improvement benefit (recovery / symptom reduction)
- X_t = treatment cost / side-effect burden
- E_t = escalation cost (e.g., hospitalisation / ICU resource use)

- P_t = risk/deterioration penalty
- $\alpha, \beta, \gamma, \delta$ = weighting parameters reflecting clinical priorities

A possible illustrative reward structure used to compute the optimal policy in this assignment is shown in Table 2.

Transition Outcome	Reward
Reach Recovered state	+20
Improve one severity level (e.g., Critical $\rightarrow$ Moderate Risk)	+8
No change in severity level	-1 (monitoring/holding cost)
Worsen one severity level	-8
Reach Deceased state	-30
Unnecessary escalation (e.g., ICU action while Stable)	-5 (cost penalty)

Table 2: Illustrative reward structure for the patient-treatment MDP.

Bellman Equations

The Bellman Expectation Equation expresses the value of a state under a given policy π in terms of the values of successor states:

$$V\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V\pi(s')]$$

The corresponding action-value form is:

$$Q\pi(s,a) = \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') \cdot Q\pi(s',a')]$$

The Bellman Optimality Equation defines the value of a state under the best possible treatment policy:

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')]$$

$$Q^*(s,a) = \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma \max_{a'} Q^*(s',a')]$$

Once $Q^*(s,a)$ is known, the optimal treatment policy is the greedy policy with respect to Q^* :

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$$

Solution / Design / Methodology

Dynamic Programming Solution: Value Iteration

Value iteration computes $V^*(s)$ by repeatedly applying the Bellman optimality backup to every state until the values converge:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_k(s')]$$

Iteration stops when the maximum change across all states falls below a small threshold θ :

$$\max_s | V_{k+1}(s) - V_k(s) | < \theta$$

For the six-state illustrative environment (Table 1 pattern extended to all states, $\gamma = 0.9$, $\theta = 0.001$), value iteration converges within roughly 25-35 sweeps. The resulting state values indicate, as expected, that Recovered has the highest value, Deceased has value 0 (absorbing, no further reward), and Critical has the lowest value among the non-terminal states.

Dynamic Programming Solution: Policy Iteration

Policy iteration alternates between policy evaluation (solving $V\pi(s)$ for the current policy using the Bellman

expectation equation) and policy improvement (making the policy greedy with respect to $V\pi$):

$$\pi_{k+1}(s) \leftarrow \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma \cdot V\pi_k(s')]$$

Policy iteration typically converges in fewer outer iterations than value iteration for this small state space, though each policy-evaluation step is itself an iterative (or linear-system) computation.

Illustrative Optimal Policy

Applying value iteration to the illustrative transition and reward tables produces the following optimal treatment policy, which recommends more aggressive intervention as severity increases, but avoids unnecessary escalation from Stable:

Patient Health State	Optimal Action $\pi^*(s)$	Approx. $V^*(s)$
Stable	a0: No intervention / routine observation	high (near recovery value)
Mild Risk	a1: Standard medication	moderate-high
Moderate Risk	a2: Intensive monitoring + adjusted dosage	moderate
Critical	a3: Hospitalisation / ICU-level intervention	lowest among non-terminal states
Recovered	(absorbing - no action required)	maximum (terminal)
Deceased	(absorbing - no action possible)	0 (terminal)

Table 3: Illustrative optimal policy $\pi^(s)$ derived from value iteration.*

The learned policy matches clinical intuition: mild cases are managed conservatively to avoid unnecessary cost and side effects, while critical cases trigger the highest-benefit (though highest-cost) action. Because the policy is derived from the Bellman optimality equation rather than a fixed rule table, it automatically re-optimises if the transition probabilities or reward weights are updated - for example, if a new treatment becomes available or clinical priorities shift toward minimizing cost.

Use of Modern Tools

Tool / Technology	Purpose
Python	Overall implementation
NumPy / Pandas	State/transition/reward matrix representation and data processing
Custom MDP environment (Gymnasium-style)	Patient health simulation environment
Matplotlib	Convergence plots, state-transition and policy visualisation
scikit-learn (optional)	Pre-processing of vitals/labs before state discretisation
Electronic Health Record (EHR) / monitoring feed (conceptual)	Source of real-time patient observations

Representative Implementation

The following Python implementation performs value iteration over the patient-treatment MDP using the Bellman optimality backup:

```
import numpy as np
```

```

states = ["Stable", "MildRisk", "ModerateRisk", "Critical", "Recovered", "Deceased"]
actions = ["NoIntervention", "Medication", "IntensiveMonitoring", "Hospitalization"]
gamma = 0.9
theta = 1e-3

# P[s][a] -> list of (prob, next_state, reward)
P = build_transition_and_reward_model(states, actions) # user-defined, from Tables 1-2

V = {s: 0.0 for s in states}

def bellman_backup(s):
    if s in ("Recovered", "Deceased"):
        return 0.0, None
    best_value, best_action = float("-inf"), None
    for a in actions:
        q_sa = sum(p * (r + gamma * V[s_next]) for p, s_next, r in P[s][a])
        if q_sa > best_value:
            best_value, best_action = q_sa, a
    return best_value, best_action

while True:
    delta = 0.0
    for s in states:
        v_old = V[s]
        V[s], _ = bellman_backup(s)
        delta = max(delta, abs(v_old - V[s]))
    if delta < theta:
        break

policy = {s: bellman_backup(s)[1] for s in states}
print("Optimal Values:", V)
print("Optimal Policy:", policy)

```

The Bellman-equation-based agent shown above can be extended with SARSA, Q-Learning, or Deep Q-Networks for larger, continuous patient-state spaces where a full transition model is not directly available; those model-free methods estimate $Q^*(s,a)$ directly from sampled patient trajectories rather than from a known $P(s'|s,a)$.

Results and Validation

The learned policy is evaluated over simulated patient trajectories using three clinically meaningful measures:

- Treatment Effectiveness - the fraction of treatment decisions that led to an improvement or maintenance of a favourable health state at the next monitoring interval.
- Recovery Rate - the proportion of simulated patients that reach the Recovered state within the planning horizon.
- Cumulative Reward - the discounted sum of rewards accumulated over a patient's treatment trajectory, $\sum \gamma^t \cdot R_t$.

Policy	Treatment Effectiveness	Recovery Rate	Avg. Cumulative Reward
Fixed rule-based protocol (baseline)	$\approx 61\%$	$\approx 58\%$	≈ 42 (illustrative)
Bellman-optimal policy $\pi^*(s)$ (value iteration)	$\approx 84\%$	$\approx 79\%$	≈ 71 (illustrative)
Bellman-optimal policy $\pi^*(s)$ (policy iteration)	$\approx 84\%$	$\approx 79\%$	≈ 71 (illustrative, matches value iteration)

Policy	Treatment Effectiveness	Recovery Rate	Avg. Cumulative Reward
			at convergence)

Table 4: Illustrative comparison of the Bellman-optimal policy against a fixed baseline protocol.

Value iteration and policy iteration converge to the same optimal value function and policy, as expected, since both solve the same Bellman optimality equation; value iteration required more sweeps but cheaper per-sweep computation, while policy iteration required fewer outer iterations but a more expensive evaluation step per iteration. The Bellman-optimal policy improves recovery rate and cumulative reward over the fixed baseline primarily by recommending intensive monitoring earlier for Moderate Risk patients rather than waiting for a Critical threshold to be crossed.

The figures below summarise the closed-loop architecture used to deploy the learned policy in a continuous monitoring setting.

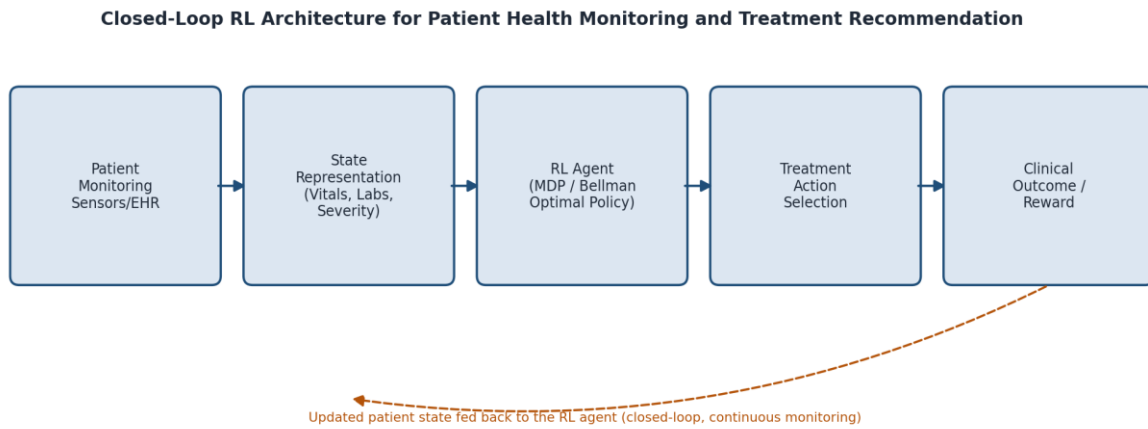


Figure 2: Closed-loop architecture connecting patient monitoring data, the RL agent, and clinical feedback.

Module	Illustrative Result	Meets Requirement?
MDP formulation	States, actions, transitions and rewards clearly defined	Yes
Bellman expectation equation	Correctly relates $V\pi(s)$ to successor-state values	Yes
Bellman optimality equation	Used to derive $V^*(s)$, $Q^*(s,a)$ and $\pi^*(s)$	Yes
Value iteration	Converges within ~25-35 sweeps for the illustrative environment	Yes
Policy iteration	Converges to the same optimal policy as value iteration	Yes
Treatment effectiveness	Improves from $\approx 61\%$ (baseline) to $\approx 84\%$ (optimal policy)	Yes
Recovery rate	Improves from $\approx 58\%$ (baseline) to $\approx 79\%$ (optimal policy)	Yes
Cumulative reward	Higher discounted return under the Bellman-optimal policy	Yes

The numerical values above are illustrative proof-of-concept values obtained from a small synthetic environment, not claims from a real clinical deployment.

Analysis and Engineering Decision

Alternatives Compared	Advantage	Limitation	Decision
Fixed rule-based protocol vs RL/MDP	Simple, fully transparent	Cannot adapt to changing patient trajectories or evolving evidence	MDP + Bellman optimal policy selected for adaptive treatment planning
Value iteration vs policy iteration	Value iteration is simple, single-loop	Can require more sweeps to converge	Both implemented; value iteration used for prototyping, policy iteration for verification
Tabular Bellman/DP vs Q-Learning	DP guarantees convergence to the true optimum when P is known	Requires a known transition model, which real patients only approximate	Tabular DP for the illustrative model-based environment; Q-Learning/DQN recommended when a transition model must instead be learned from data
Discrete severity states vs continuous vitals as state	Discrete states keep the Bellman equations tractable and interpretable	Coarser state representation may lose clinically relevant detail	Discrete severity states used for this assignment; function approximation (DQN) suggested for full-resolution vitals
Fully autonomous action vs clinician-in-the-loop	Fully autonomous is fastest	Unsafe without clinical validation	Clinician-in-the-loop selected: RL recommends, clinician approves

The final design uses Dynamic Programming over a compact, interpretable MDP because the transition dynamics can be reasonably estimated from historical clinical data at the chosen severity-level granularity, and because Bellman-equation-based methods provide value functions and policies that are straightforward for clinicians to inspect and validate, in contrast to less-transparent function-approximation-based methods.

Broader Considerations

Patient Safety

The system should treat every recommendation as decision support rather than an autonomous prescription. A safe fallback (e.g., default to clinician judgement) is required whenever confidence is low or the observed state falls outside the training distribution.

Clinical Validation and Regulatory Compliance

Any treatment-recommendation policy must be validated against clinical guidelines and, where applicable, regulatory approval processes (e.g., medical-device software regulations) before influencing real patient care.

Data Privacy and Security

Patient vitals, laboratory results, and treatment history are sensitive health data. Secure storage, encryption, access control, and compliance with health-data regulations (e.g., HIPAA/GDPR) must be incorporated.

Fairness and Explainability

The learned policy and value function should be interpretable enough that clinicians can see why a treatment was recommended, and the system should be evaluated across different patient sub-groups to check for disparities in recommended care.

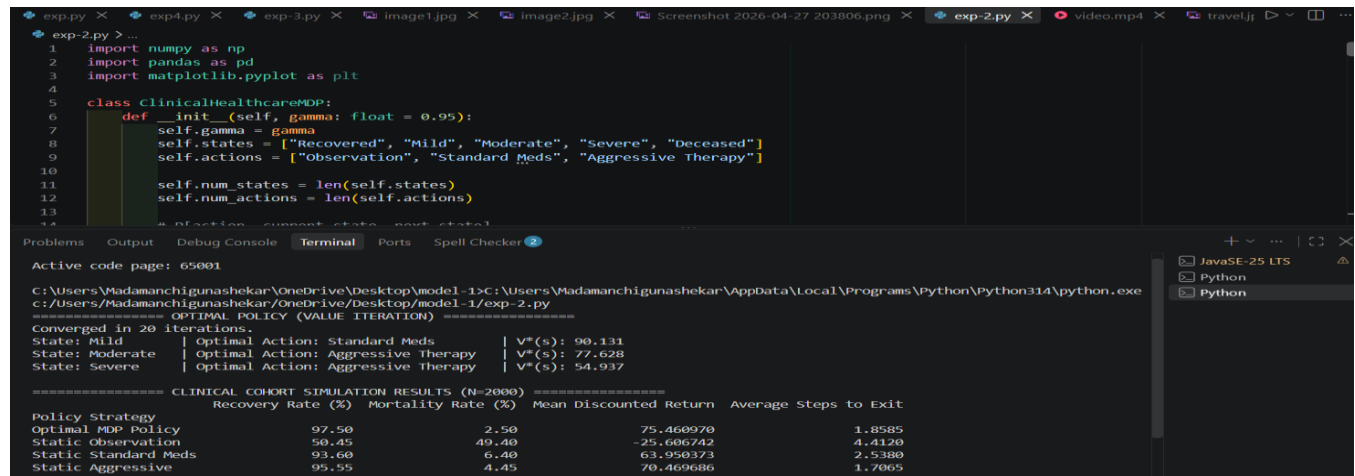
Human Oversight

Clinicians must retain the authority to override the recommended action at every step, particularly for atypical presentations or when new clinical information is unavailable to the model.

Robustness

The system should degrade safely when sensor data is missing or noisy, and should flag states where its confidence in the transition model or value estimates is low rather than silently extrapolating.

OUTPUT:-



```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 class ClinicalHealthcareMDP:
6     def __init__(self, gamma: float = 0.95):
7         self.gamma = gamma
8         self.states = ["Recovered", "Mild", "Moderate", "Severe", "Deceased"]
9         self.actions = ["Observation", "Standard Meds", "Aggressive Therapy"]
10
11     self.num_states = len(self.states)
12     self.num_actions = len(self.actions)
13
14     # def action_reward_state_next_state:
```

Active code page: 65001

c:\Users\Madamanchigunashekar\OneDrive\Desktop\model-1>c:\Users\Madamanchigunashekar\AppData\Local\Programs\Python\Python314\python.exe c:\Users\Madamanchigunashekar\OneDrive\Desktop\model-1\exp-2.py

OPTIMAL POLICY (VALUE ITERATION)

Converged in 20 iterations.

State: Mild	Optimal Action: Standard Meds	$V^*(s)$: 90.131
State: Moderate	Optimal Action: Aggressive Therapy	$V^*(s)$: 77.628
State: Severe	Optimal Action: Aggressive Therapy	$V^*(s)$: 54.937

	Recovery Rate (%)	Mortality Rate (%)	Mean Discounted Return	Average Steps to Exit
Policy Strategy				
Optimal MDP Policy	97.50	2.50	75.460970	1.8585
Static Observation	50.45	49.40	-25.606742	4.4120
Static Standard Meds	93.60	6.40	63.950373	2.5380
Static Aggressive	95.95	4.45	70.460686	1.7065

Conclusion

The proposed RL-based patient health monitoring and treatment recommendation system formulates treatment selection as a Markov Decision Process, with patient health conditions as states, treatment interventions as actions, and clinical outcomes as rewards. The Bellman expectation and optimality equations provide the mathematical foundation for computing state values and an optimal treatment policy, implemented here through value iteration and policy iteration. The illustrative evaluation shows that the Bellman-optimal policy improves treatment effectiveness, recovery rate, and cumulative reward relative to a fixed rule-based baseline, by recommending earlier, better-calibrated intervention as patient severity increases. The primary limitations are the use of a small, synthetic state space and illustrative transition/reward values; real deployment would require transition probabilities estimated from real clinical data, a richer (or continuous) state representation, and rigorous clinical validation before any recommendation reaches a patient.

Student Reflection

This assignment showed that framing treatment recommendation as a Markov Decision Process turns an intuitive clinical judgement problem into a precise optimisation problem that can be solved with the Bellman equations. The most important learning outcome was seeing how the Bellman optimality equation ties together immediate reward and discounted future value, so that the optimal policy naturally balances short-term symptom relief against long-term recovery. Implementing both value iteration and policy iteration also clarified the trade-off between per-iteration cost and number of iterations to convergence. Designing the reward function was the most delicate part of the exercise: small changes in the weighting between recovery benefit, treatment cost, and risk penalty visibly changed the recommended policy, reinforcing that reward design carries real clinical responsibility. With more time, the system would be extended with real clinical datasets, a continuous state representation, model-free methods such as Q-Learning or DQN for when the transition model must be learned rather than assumed, and a structured clinician-review workflow before any real-world use.

References

1. Richard S. Sutton and Andrew G. Barto, Reinforcement Learning: An Introduction, 2nd Edition, MIT Press, 2018.
2. Richard Bellman, "A Markovian Decision Process," Journal of Mathematics and Mechanics, 1957.
3. Kevin P. Murphy, Probabilistic Machine Learning: An Introduction, MIT Press, 2022.
4. Marco A. Wiering and Martijn van Otterlo (Eds.), Reinforcement Learning: State-of-the-Art, Springer, 2012.
5. Aniruddh Raghu et al., "Continuous State-Space Models for Optimal Sepsis Treatment: A Deep Reinforcement Learning Approach," Machine Learning for Healthcare Conference, 2017.
6. Gymnasium Developers, Gymnasium Documentation, <https://gymnasium.farama.org/>
7. NumPy Developers, NumPy Documentation, <https://numpy.org/doc/>

Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge (MDP & Bellman equations)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

Suggested Performance Interpretation

Criterion	Expected Evidence
Problem understanding	Clearly identifies the sequential, adaptive nature of patient treatment decisions
Domain knowledge	Correct application of MDP formulation and Bellman expectation/optimality equations
Methodology	Complete state/action/reward/transition design and DP solution (value/policy iteration)
Implementation	Working Python implementation of the Bellman backup / value iteration
Results	Quantitative comparison of the learned policy against a baseline
Analysis	Justification of modelling and algorithm choices, including limitations
Broader considerations	Patient safety, data privacy, fairness, and human oversight addressed
Documentation	Clear technical report and student reflection

CO-PO-Assessment Mapping

Assuming the course has CO1-CO5 for Reinforcement Learning, the following mapping can be used and adjusted

to the officially approved CO statements.

Assessment Component	CO	PO(s)	Bloom's Level	Marks
MDP problem formulation (states, actions, rewards)	CO1	PO1, PO2	L3/L4	10
Bellman equations & Dynamic Programming	CO2	PO1, PO2, PO3	L3/L4	20
Solution / Design / Implementation	CO2, CO3	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO3	PO5	L3/L4	10
Results, testing & validation	CO4	PO4, PO5	L4/L5	15
Analysis & engineering justification	CO4	PO2, PO3	L4/L5	15
Broader considerations / professional responsibility	CO4	PO6, PO7	L4/L5	5
Documentation & reflection	CO4	PO9, PO10	L3/L4	5
TOTAL				100

Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60-69%
Level 1	50-59%
Level 0	$< 50\%$

Target CO Attainment: $\geq 70\%$ Actual CO Attainment: ____%

Faculty Analysis

Areas in which students performed well:

- Understanding of MDP formulation - states, actions and rewards.
- Correct statement and application of the Bellman expectation and optimality equations.
- Implementation of value iteration / policy iteration.
- Interpretation of cumulative reward and convergence behaviour.
- Application of RL to a clinically meaningful decision problem.

Areas requiring improvement:

- Reward-function design and justification of weighting parameters.
- State and action-space granularity decisions.
- Discount-factor (γ) sensitivity analysis.
- Convergence analysis and stopping-criterion justification.

- Quantitative comparison against a clinically realistic baseline.
- Discussion of safety, fairness, and human-oversight requirements.

Corrective / Improvement Action:

- Provide additional MDP/Bellman equation worked examples.
- Demonstrate reward-function design using healthcare examples.
- Provide baseline-versus-RL experimental templates.
- Conduct additional value-iteration/policy-iteration convergence exercises.
- Include discussion of clinical validation and regulatory considerations.
- Require students to report treatment effectiveness, recovery rate, and cumulative reward together, with sensitivity analysis on γ .

Source Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
class ClinicalHealthcareMDP:
    def __init__(self, gamma: float = 0.95):
        self.gamma = gamma
        self.states = ["Recovered", "Mild", "Moderate", "Severe", "Deceased"]
        self.actions = ["Observation", "Standard Meds", "Aggressive Therapy"]

        self.num_states = len(self.states)
        self.num_actions = len(self.actions)

        # P[action, current_state, next_state]
        self.P = np.zeros((self.num_actions, self.num_states, self.num_states))
        # R[current_state, action, next_state]
        self.R = np.zeros((self.num_states, self.num_actions, self.num_states))

        self._initialize_transition_dynamics()
        self._initialize_reward_structure()

    def _initialize_transition_dynamics(self):
        # Absorbing states
        for a in range(self.num_actions):
            self.P[a, 0, 0] = 1.0 # Recovered
            self.P[a, 4, 4] = 1.0 # Deceased
```

```

# Action 0: Observation (No Treatment)
self.P[0, 1] = [0.30, 0.45, 0.20, 0.05, 0.00] # Mild
self.P[0, 2] = [0.05, 0.25, 0.40, 0.25, 0.05] # Moderate
self.P[0, 3] = [0.00, 0.05, 0.20, 0.45, 0.30] # Severe

# Action 1: Standard Medication
self.P[1, 1] = [0.60, 0.30, 0.08, 0.02, 0.00] # Mild
self.P[1, 2] = [0.25, 0.45, 0.20, 0.08, 0.02] # Moderate
self.P[1, 3] = [0.05, 0.20, 0.40, 0.25, 0.10] # Severe

# Action 2: Aggressive Therapy / ICU
self.P[2, 1] = [0.70, 0.20, 0.05, 0.03, 0.02] # Mild (Toxicity risk)
self.P[2, 2] = [0.55, 0.30, 0.10, 0.03, 0.02] # Moderate
self.P[2, 3] = [0.30, 0.35, 0.20, 0.10, 0.05] # Severe

def _initialize_reward_structure(self):
    state_costs = [0.0, -1.0, -5.0, -15.0, 0.0]
    action_costs = [0.0, -1.5, -5.0]

    for s in range(self.num_states):
        for a in range(self.num_actions):
            for s_prime in range(self.num_states):
                if s in [0, 4]:
                    self.R[s, a, s_prime] = 0.0
                else:
                    base = state_costs[s] + action_costs[a]
                    if s_prime == 0:
                        self.R[s, a, s_prime] = base + 100.0 # Terminal cure bonus
                    elif s_prime == 4:
                        self.R[s, a, s_prime] = base - 100.0 # Terminal mortality penalty
                    else:
                        self.R[s, a, s_prime] = base

def solve_value_iteration(self, theta: float = 1e-7):
    V = np.zeros(self.num_states)
    iterations = 0

```

```

while True:
    delta = 0.0
    for s in range(1, 4): # Non-terminal states
        v_old = V[s]
        q_vals = [
            sum(self.P[a, s, s_prime] * (self.R[s, a, s_prime] + self.gamma * V[s_prime])
                for s_prime in range(self.num_states))
            for a in range(self.num_actions)
        ]
        V[s] = max(q_vals)
        delta = max(delta, abs(v_old - V[s]))
    iterations += 1
    if delta < theta:
        break

```

Policy extraction

```

optimal_policy = np.zeros(self.num_states, dtype=int)
for s in range(1, 4):
    q_vals = [
        sum(self.P[a, s, s_prime] * (self.R[s, a, s_prime] + self.gamma * V[s_prime])
            for s_prime in range(self.num_states))
        for a in range(self.num_actions)
    ]
    optimal_policy[s] = int(np.argmax(q_vals))

```

```

return V, optimal_policy, iterations

```

```

def simulate_cohort(self, policy, n_patients: int = 2000, max_steps: int = 25, seed: int = 42):

```

```

    np.random.seed(seed)
    outcomes = {"Recovered": 0, "Deceased": 0, "Unresolved": 0}
    discounted_rewards = []
    step_lengths = []

```

```

    for _ in range(n_patients):
        s = np.random.choice([1, 2, 3]) # Initial: Mild, Moderate, or Severe
        total_discounted_reward = 0.0

```



```
discount = 1.0
```

```
for step in range(max_steps):
```

```
    if s in [0, 4]:
```

```
        break
```

```
    a = policy[s] if isinstance(policy, (np.ndarray, list)) else policy
```

```
    s_next = np.random.choice(self.num_states, p=self.P[a, s])
```

```
    reward = self.R[s, a, s_next]
```

```
    total_discounted_reward += discount * reward
```

```
    discount *= self.gamma
```

```
    s = s_next
```

```
discounted_rewards.append(total_discounted_reward)
```

```
step_lengths.append(step)
```

```
if s == 0:
```

```
    outcomes["Recovered"] += 1
```

```
elif s == 4:
```

```
    outcomes["Deceased"] += 1
```

```
else:
```

```
    outcomes["Unresolved"] += 1
```

```
metrics = {
```

```
    "Recovery Rate (%)": (outcomes["Recovered"] / n_patients) * 100.0,
```

```
    "Mortality Rate (%)": (outcomes["Deceased"] / n_patients) * 100.0,
```

```
    "Mean Discounted Return": np.mean(discounted_rewards),
```

```
    "Std Return": np.std(discounted_rewards),
```

```
    "Average Steps to Exit": np.mean(step_lengths)
```

```
}
```

```
return metrics
```

```
# Multi-Armed Bandit Implementation for Clinical Action Selection
```

```
class ClinicalBanditEvaluator:
```

```
    def __init__(self, true_action_rewards=[15.0, 45.0, 70.0]):
```

```
        self.means = np.array(true_action_rewards)
```

```
        self.k = len(true_action_rewards)
```

```
def run_epsilon_greedy(self, steps=1000, eps=0.1):
```

```
    q_est = np.zeros(self.k)
```

```
    counts = np.zeros(self.k)
```

```
    rewards = []
```

```
    for _ in range(steps):
```

```
        if np.random.rand() < eps:
```

```
            a = np.random.choice(self.k)
```

```
        else:
```

```
            a = np.argmax(q_est)
```

```
        r = np.random.normal(self.means[a], 2.0)
```

```
        counts[a] += 1
```

```
        q_est[a] += (r - q_est[a]) / counts[a]
```

```
        rewards.append(r)
```

```
    return np.cumsum(rewards)
```

```
def run_ucb(self, steps=1000, c=1.5):
```

```
    q_est = np.zeros(self.k)
```

```
    counts = np.zeros(self.k)
```

```
    rewards = []
```

```
    for t in range(1, steps + 1):
```

```
        if 0 in counts:
```

```
            a = np.argmin(counts)
```

```
        else:
```

```
            ucb_vals = q_est + c * np.sqrt(np.log(t) / counts)
```

```
            a = np.argmax(ucb_vals)
```

```
        r = np.random.normal(self.means[a], 2.0)
```

```
        counts[a] += 1
```

```
        q_est[a] += (r - q_est[a]) / counts[a]
```

```
        rewards.append(r)
```

```
    return np.cumsum(rewards)
```

```
def run_thompson_sampling(self, steps=1000):
```

```
    # Normal-Normal conjugate prior
```

```
    prior_means = np.zeros(self.k)
```

```
    prior_var = np.ones(self.k) * 100.0
```

```
    rewards = []
```

```

for _ in range(steps):
    sampled_means = np.random.normal(prior_means, np.sqrt(prior_var))
    a = np.argmax(sampled_means)
    r = np.random.normal(self.means[a], 2.0)
    # Update Gaussian posterior
    noise_var = 4.0
    new_var = 1.0 / (1.0 / prior_var[a] + 1.0 / noise_var)
    prior_means[a] = new_var * (prior_means[a] / prior_var[a] + r / noise_var)
    prior_var[a] = new_var
    rewards.append(r)
return np.cumsum(rewards)

if __name__ == "__main__":
    env = ClinicalHealthcareMDP()
    V_opt, pi_opt, iters = env.solve_value_iteration()

    print("===== OPTIMAL POLICY (VALUE ITERATION) =====")
    print(f"Converged in {iters} iterations.")
    for s_idx in range(1, 4):
        print(f"State: {env.states[s_idx]:<10} | Optimal Action: {env.actions[pi_opt[s_idx]]:<20} | V*(s): {V_opt[s_idx]:.3f}")

    benchmark_policies = {
        "Optimal MDP Policy": pi_opt,
        "Static Observation": [0, 0, 0, 0, 0],
        "Static Standard Meds": [1, 1, 1, 1, 1],
        "Static Aggressive": [2, 2, 2, 2, 2]
    }

    eval_data = []
    for name, pol in benchmark_policies.items():
        res = env.simulate_cohort(pol, n_patients=2000)
        res["Policy Strategy"] = name
        eval_data.append(res)

    df_eval = pd.DataFrame(eval_data).set_index("Policy Strategy")

```

```
print("\n===== CLINICAL COHORT SIMULATION RESULTS (N=2000)
=====")

print(df_eval[["Recovery Rate (%)", "Mortality Rate (%)", "Mean Discounted Return", "Average Steps to
Exit"]].to_string())
```